

KT그룹

AX 전략기획 및 멀티에이전트 서비스 개발

Agentic AI LLMOps



CONTENTS

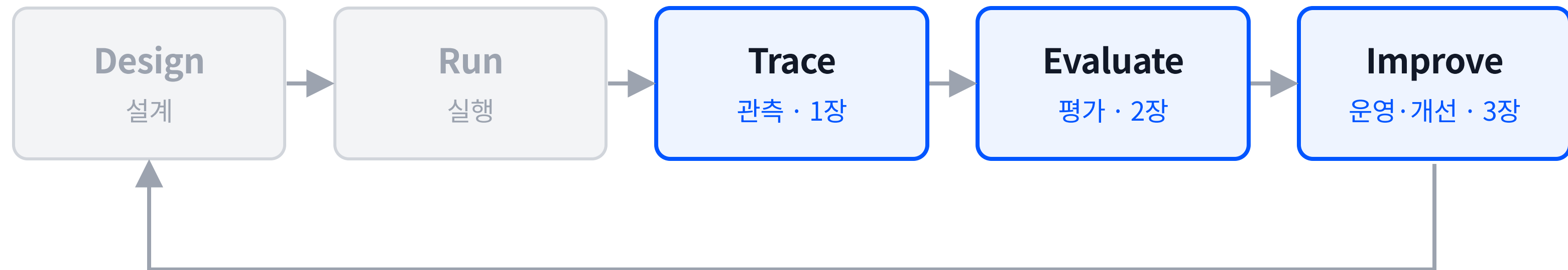
목차

- 01 LLM 서비스 관측성(Observability)
- 02 품질 평가(Evaluation) 설계
- 03 비용·모델 운영 전략

과정 전체 지도



운영의 다섯 단계와 이 강의가 다루는 세 단계



개선 결과는 다시 설계로 · 다섯 단계는 순환

운영 대상 = 앞서 만든 FAQ 안내 서비스(RAG)

이 강의에서 쓰는 호출 경로 · 실행 상태 최소 지식

호출 경로 (call path)

학생 코드가 litellm 함수에 모델 이름을 넘기고, litellm이 그 이름의 프로바이더 API를 직접 부르는 구조.

```
코드 → litellm SDK → 모델
```

completion (litellm)

모델 · 메시지 · 도구 목록을 넘기면 응답 객체가 돌아오는 단일 함수.

```
completion(model=,  
messages=, tools=)
```

채팅 모델 객체 (init_chat_model)

같은 모델을 랭체인 객체로 감싼 형태. 프로바이더 이름이 litellm.

```
init_chat_model(..., model_provider="litellm")
```

구조화 출력 (Structured Output)

pydantic 스키마를 넘기면 응답이 그 형식으로 강제되어 파싱이 필요 없음.

```
llm.with_structured_output(Schema)
```

실행 상태(State) · 노드

그래프는 노드를 하나씩 지나가고, 노드마다 갱신되는 값 묶음이 실행 상태(State).

```
def node(state): return  
{...}
```

실행 상태 저장 (MemorySaver)

그래프 실행의 각 단계 상태를 저장해 두는 장치. 저장된 상태가 있어야 이어가기 · 디버깅이 성립.

```
StateGraph(...).compile(checkpointer=MemorySaver())
```

LLMOps

LLM 서비스 관측성(Observability)

보이지 않는 것 / 트레이스 / 활성화 / 구간 분해 / 태깅

학습 목표

- 운영 중인 서비스에 트레이스(trace)를 켜고, 호출 트리에서 지연·비용·오류가 생긴 런(run)을 수치와 함께 짚어 낼 수 있습니다.
- 메타데이터·태그를 붙여 기능 단위로 비용과 지연을 묶어 보고, ls_model_name 메타데이터가 없을 때 비용 열이 비는 이유를 설명할 수 있습니다.

▶ LLM 서비스 관측성

품질 평가 설계

비용·모델 운영 전략

문제 상황: 응답은 왔는데 안이 보이지 않는 서비스

```
$ curl -X POST /ask -d '{"question":  
  "자유이용권 환불이 되나요?"}'  
  
{  
  "answer": "네, 자유이용권은 환불 가능합니다.  
  - 이용일 전일까지 취소: 전액 환불  
  - 이용일 당일 취소: 50% 환불  
  - 이용 개시 후: 환불 불가"}  
}
```

(응답까지 3.6초)

질문	답
3.6초 중 어느 구간이 느렸나	알 수 없음
모델을 몇 번 불렀고 토큰은 몇이었나	알 수 없음
검색 도구가 실제로 호출됐나	알 수 없음
이 한 번의 응답에 든 비용	알 수 없음

기록이 없는 서비스는 "느린 것 같다"까지만 말할 수 있음.

트레이스(trace): 요청 한 건이 지나간 모든 단계의 완전한 기록
런(run): 그 안의 단계 하나. 모델 호출, 도구 실행, 함수 호출
각각

런에는 유형(LLM · Chain · Tool · Retriever), 시작·종료 시각, 입력·출력, 토큰 수가 붙음. 런들이 부모-자식으로 이어져 트리가 됨.

프로젝트(project)는 한 서비스의 트레이스를 모아 담는 통. 피드백(feedback)은 런 하나에 붙이는 점수.

같은 trace라는 단어가 분산 시스템 계보에서는 여러 서비스를 가로지르는 요청 경로를 가리킴. 여기서는 요청 한 건 안의 단계들.

같은 기록의 두 관점: 호출 트리와 워터폴

호출 트리 (누가 누구를 불렀나)

```
chain rag_agent 2.90s 543 tok
├─ llm litellm.completion 1.38s 182 tok
├─ tool faq_search 0.14s
└─ llm litellm.completion 1.38s 361 tok
```

부모 런 아래 자식 런. 도구 호출 루프가 그대로 트리가 됨.

워터폴 (언제 얼마나 걸렸나)

```
chain 2.90s
llm 1.38s
tool 0.14s
llm 1.38s
```

같은 런을 시간축에 편 것. 가장 긴 막대가 지연 구간.

문제상황과 목표

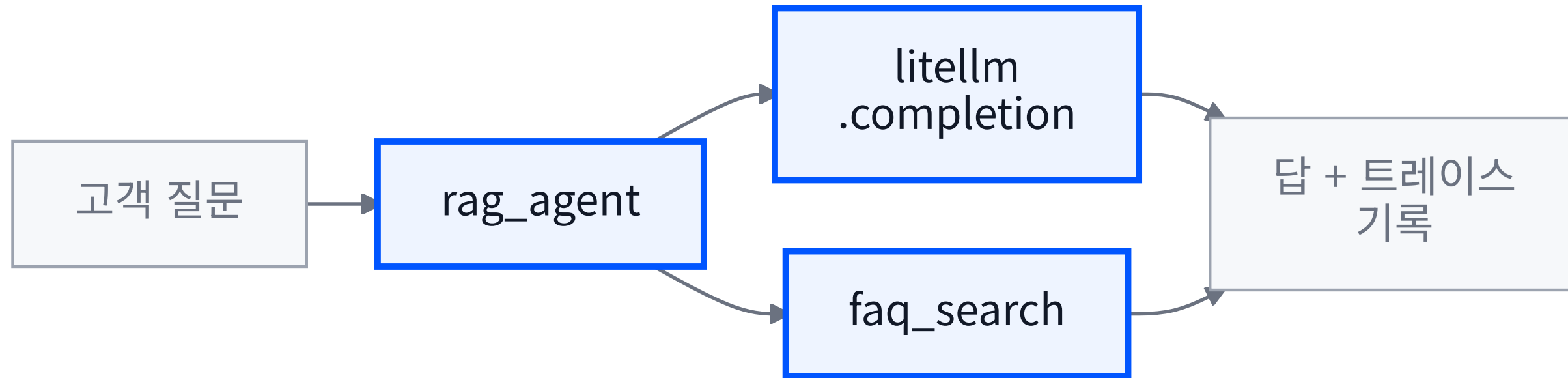
문제

구름월드(테마파크) 고객센터 담당자: "환불 문의 하나에 답이 4초나 걸리는데, 어디서 시간이 걸리고 한 건에 얼마가 드는지 아무도 몰라요 ㅠㅠ"

목표

구름월드 FAQ 안내 서비스가 고객 문의 하나를 처리하며 거치는 단계(질문 이해 → 안내 자료 검색 → 답 작성)를, **단계마다 걸린 시간과 비용까지** 한눈에 보이게 만들어봅시다.

각 단계별 요구사항



노드	무엇을 받아	어떻게	무엇을 넘긴다
rag_agent	고객 질문 한 줄을	안내 서비스를 그대로 돌려되, 그 한 번의 처리 전체를 하나의 기록 묶음으로 감싸	답을 넘긴다
litellm.completion	서비스가 모델에 던지는 호출을	어느 모델을 썼는지 표지를 달아, 시간·토큰·비용이 남는 기록으로 감싸	모델의 응답을 그대로 서비스에 돌려준다
faq_search	서비스가 FAQ 문서를 찾아보는 검색을	검색마다 걸린 시간이 남는 기록으로 감싸	검색 결과를 그대로 돌려준다

관측하려는 서비스의 안: 모델에 무엇이 들어가나

```
# 고정 지침(시스템 프롬프트)
# 서비스가 모델에 항상 주는 프롬프트
너는 시설 안내 담당자다.
사실 질문에는 faq_search 도구로 근거를 찾아 답한다.
인사말 등 검색이 필요 없는 말에는 바로 답한다.
근거가 없으면 모른다고 답한다.
```

```
# 도구: 모델이 부를 수 있는 검색 도구의 설명
faq_search: 시설 FAQ 문서에서 질문과 관련된
청크를 검색한다. 시설 이용, 요금, 환불,
운영 관련 질문에 쓴다.
```

모델에 들어가는 것은 **시스템 프롬프트 + 도구 + 대화 이력 + 근거 자료**의 네 구획. 강조한 두 줄이 서비스의 동작을 결정함: 검색을 시키는 줄이 도구 호출을 만들고, 모른다고 답하라는 줄이 답의 근거를 만듦.

```
# 대화 이력: 이번 턴의 사용자 질문
자유이용권 환불이 되나요?
```

```
# 근거 자료: 도구가 찾아오면 뒤에 붙는 FAQ 청크
[티켓] Q: 자유이용권 환불 규정이 어떻게 되나요?
A: 이용일 전날까지 취소하면 전액 환불...
```

```
# 정상이라면
모델 → faq_search("자유이용권 환불") 호출
→ 검색 결과를 근거로 답 작성
답: 이용일 전날까지 전액, 당일 50%, 개시 후 불가
```

```
# 실제 (관측 전)
답은 같은 문장이 옴. 3.6초.
검색을 했는지, 모델을 몇 번 불렀는지,
어디서 3.6초가 걸렸는지는 보이지 않음
```

1단계: 환경변수로 트레이스 활성화

```
import os, litellm, day05_rag as d5
from langsmith import traceable
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_PROJECT"] = "day08"
db = d5.open_db()
_comp = litellm.completion
litellm.completion = traceable(run_type="llm",
                               name="litellm.completion",
                               metadata={"ls_model_name": "gpt-5.6-luna",
                                         "ls_provider": "openai"})(_comp)
_make = d5.make_search_tool
d5.make_search_tool = lambda db: traceable(
    run_type="tool", name="faq_search")(_make(db))
@traceable(run_type="chain", name="rag_agent")
def ask(question: str) -> str:
    return d5.run_rag_agent(question, db)
print(ask("자유이용권 환불이 되나요?")[:80])
```



LANGSMITH_TRACING이 "true"면 이후 모든 런이 전송됨. 키는 **LANGSMITH_API_KEY**, 프로젝트는 **LANGSMITH_PROJECT**.

서비스 코드는 바뀌지 않음. 켜는 것은 환경변수 2개. 키 **LANGSMITH_API_KEY**도 **OPENAI_API_KEY**처럼 실행 환경에 준비해 두면 **day05_rag**를 import할 때 함께 읽힘.

2단계: 모델 호출을 llm 런으로 기록

```
import os, litellm, day05_rag as d5
from langsmith import traceable
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_PROJECT"] = "day08"
db = d5.open_db()
_comp = litellm.completion
litellm.completion = traceable(run_type="llm",
                               name="litellm.completion",
                               metadata={"ls_model_name": "gpt-5.6-luna",
                                         "ls_provider": "openai"})(_comp)
_make = d5.make_search_tool
d5.make_search_tool = lambda db: traceable(
    run_type="tool", name="faq_search")(_make(db))
@traceable(run_type="chain", name="rag_agent")
def ask(question: str) -> str:
    return d5.run_rag_agent(question, db)
print(ask("자유이용권 환불이 되나요?")[:80])
```



traceable(run_type="llm") 래퍼 함수의 호출 한 번 = llm 런 하나. 랭체인 객체가 아닌 직접 호출도 이렇게 기록됨.

ls_model_name · ls_provider 메타데이터가 있어야 토큰 수가 비용으로 환산됨. 없으면 비용 열이 빈 채로 기록됨.

3단계: 도구 호출을 tool 런으로 기록

```
import os, litellm, day05_rag as d5
from langsmith import traceable
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_PROJECT"] = "day08"
db = d5.open_db()
_comp = litellm.completion
litellm.completion = traceable(run_type="llm",
                               name="litellm.completion",
                               metadata={"ls_model_name": "gpt-5.6-luna",
                                         "ls_provider": "openai"})(_comp)
_make = d5.make_search_tool
d5.make_search_tool = lambda db: traceable(
    run_type="tool", name="faq_search")(_make(db))
@traceable(run_type="chain", name="rag_agent")
def ask(question: str) -> str:
    return d5.run_rag_agent(question, db)
print(ask("자유이용권 환불이 되나요?")[:80])
```

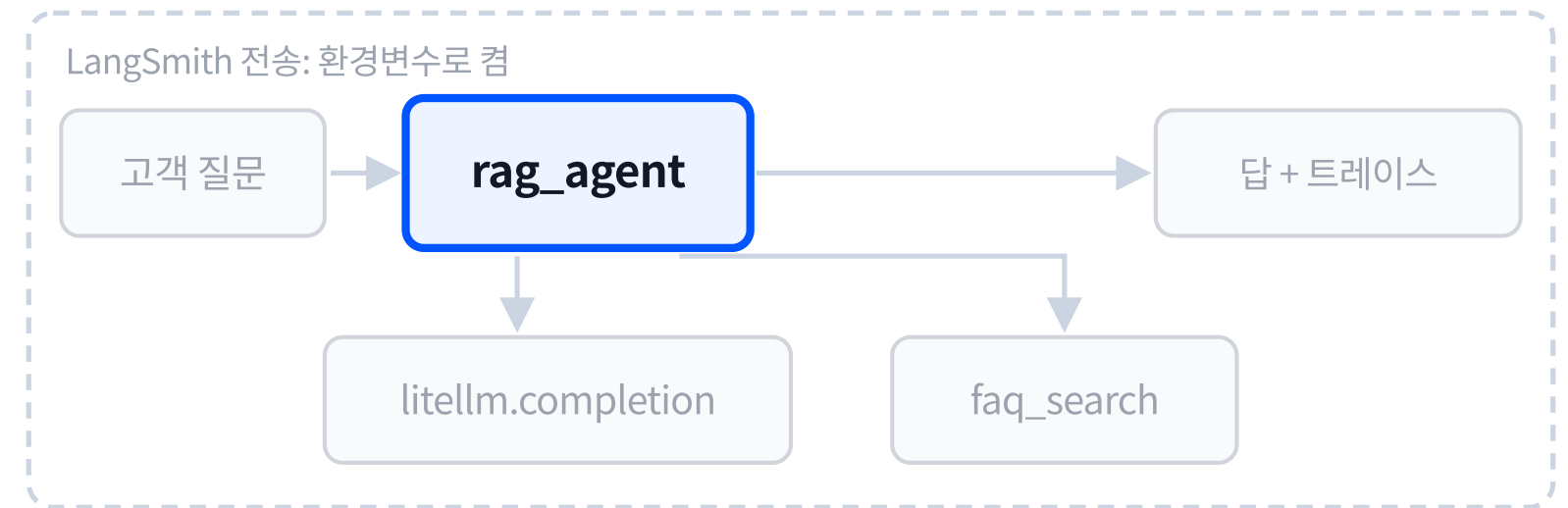


run_type="tool" 런은 토큰이 0이고 시간만 기록됨. 검색 도구가 실제로 불렸는지, 얼마나 걸렸는지가 여기서 보임.

도구 함수가 모델 호출 사이에 끼면 트리에서 llm → tool → llm 순서로 자식이 됨.

4단계: 진입 함수의 부모 런 승격

```
import os, litellm, day05_rag as d5
from langsmith import traceable
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_PROJECT"] = "day08"
db = d5.open_db()
_comp = litellm.completion
litellm.completion = traceable(run_type="llm",
                               name="litellm.completion",
                               metadata={"ls_model_name": "gpt-5.6-luna",
                                         "ls_provider": "openai"})(_comp)
_make = d5.make_search_tool
d5.make_search_tool = lambda db: traceable(
    run_type="tool", name="faq_search")(_make(db))
@traceable(run_type="chain", name="rag_agent")
def ask(question: str) -> str:
    return d5.run_rag_agent(question, db)
print(ask("자유이용권 환불이 되나요?")[:80])
```



run_type="chain" 런이 부모가 되고, 그 안에서 불린 llm·tool 런이 자식으로 매달림. 요청 한 건 = 트레이스 한 건.

함수 이름이 화면의 런 이름이 됨. 데코레이터가 진입점을 트리의 뿌리로 만들.

완성 모습: 질문 하나가 남기는 트리

"자유이용권 환불이 되나요?"

```
# LangSmith 트레이스 화면을 옮겨 적음
chain rag_agent          3.58s 575 tok
├─ llm litellm.completion 1.51s 185 tok
├─ tool faq_search        0.16s
└─ llm litellm.completion 1.85s 390 tok
```

모델이 검색을 요청하고, 결과를 받아 다시 답함, 호출 2회.

"안녕하세요!"

```
# LangSmith 트레이스 화면을 옮겨 적음
chain rag_agent          1.31s
└─ llm litellm.completion 161 tok
```

검색 없이 바로 답함, 호출 1회. 도구 런이 없는 것이 그 증거.

랭스미스 화면: 질문 하나의 트레이스

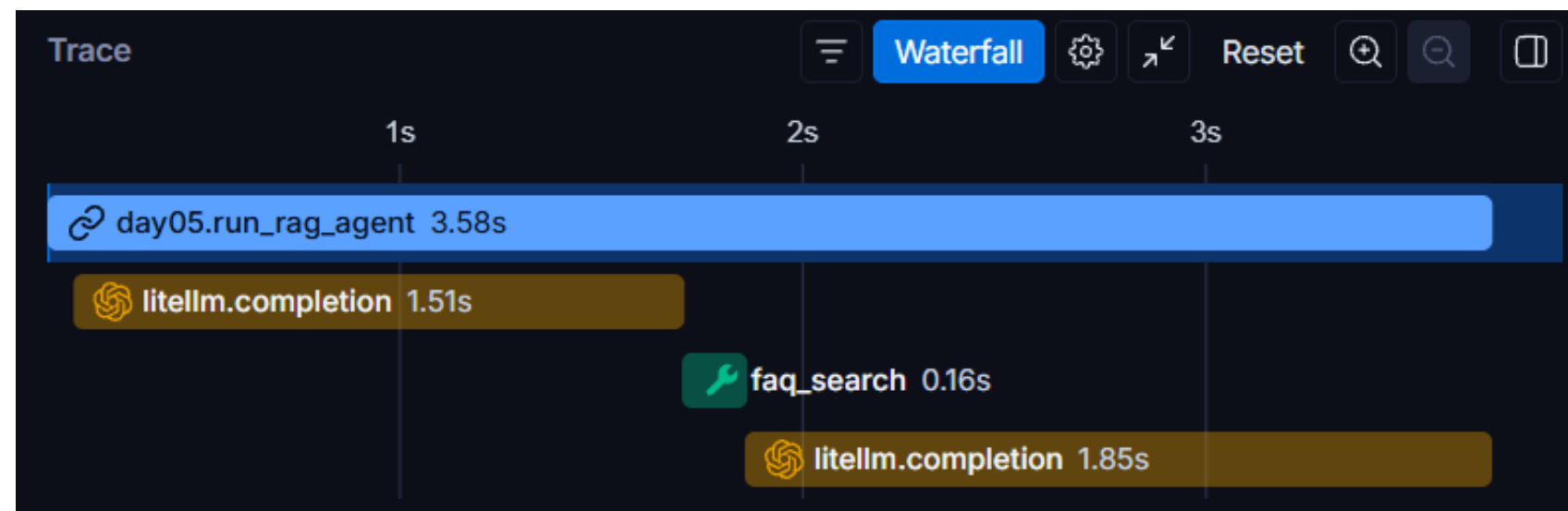
The screenshot displays the LangSmith interface for a specific trace. On the left, a vertical list of steps is shown: 'day05.run_rag_agent' (2:52 AM, 3.58s, 575 / \$0.0003), 'litellm.completion' (gpt-5.6-luna, 1.51s, 185), 'faq_search' (0.16s), and another 'litellm.completion' (gpt-5.6-luna, 1.85s, 390). The main panel on the right is titled 'day05.run_rag_agent' and includes an 'Add to Annotation Queue' button. It features tabs for 'Feedback', 'Input', 'Output', and 'Attributes'. The 'Feedback' tab is active, showing a '+ Add feedback' button. The 'Input' tab shows a 'Fields' section with a 'question' field containing the text '자유이용권 환불이 되나요?'. The 'Output' tab shows a 'Fields' section with an 'output' field containing the text '네. 자유이용권은 이용일 전날까지 취소하면 전액 환불되며, 이용일 당일 취소 시 50% 환불됩니다. 단, 이용을 시작한 후에는 환불되지 않습니다.'.

구간 분해 1: 가장 느린 런

LangSmith 트레이스 화면을 옮겨 적음

```
chain rag_agent          3.58s 575 tok
├─ llm  litellm.completion 1.51s 185 tok
├─ tool faq_search         0.16s
└─ llm  litellm.completion 1.85s 390 tok
```

읽는 순서	무엇을 보나
① 부모 런의 총 시간	3.58초, 사용자가 기다린 시간
② 자식 런 중 가장 긴 막대	두 번째 모델 호출 1.85초. 검색(0.16초)이 아니라 검색 결과를 받은 뒤의 답 생성이 가장 김
③ 그 런의 입력·출력	어떤 입력으로 호출했고 무엇이 돌아왔나



구간 분해 2: 가장 비싼 런

```
# LangSmith 트레이스 화면을 옮겨 적음
chain rag_agent      3.58s 575 tok $0.000208
├─ llm completion    1.51s 185 tok $0.000068
├─ tool faq_search    0.16s
└─ llm completion    1.85s 390 tok $0.000140
```

읽는 순서	무엇을 보나
① 토큰 수가 큰 llm 런	두 번째 호출 390 토큰, 검색 결과가 입력에 얹혀 커짐
② 비용 열	토큰 × 모델 단가. ls_model_name 메타데이터가 있는 런에만 계산됨
③ 부모 런의 합계	요청 한 건의 비용, 청구서의 최소 단위

Personal / Tracing / kt10-day08-observe2

kt10-day08-observe2 ID Tracing Evaluators Automations Insights Engine New

Default View Last 1 day Threads Traces Runs Input Search input content

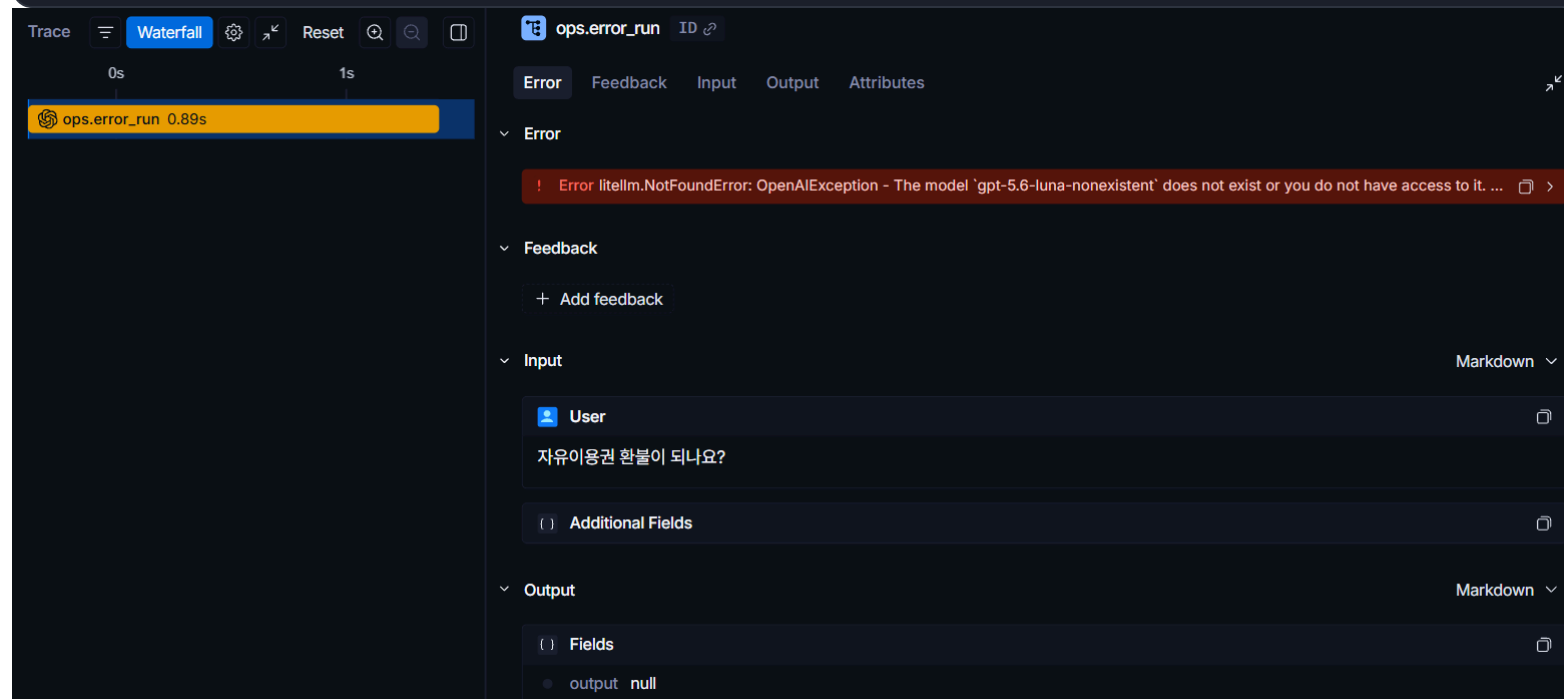
Add filter

	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation ...	Tokens	Cost	First Token	Tags	Metadata
	day05.run_rag_agent	야간 개장은 몇 시까지인가요?	야간 개장은 성수기(7~8월)...		8/27/2026, 2:5...	3.53s			545	\$0.000184			LANGSMITH_E
	day05.run_rag_agent	안녕하세요!	안녕하세요! 무엇을 도와드릴...		8/27/2026, 2:5...	1.31s			161	\$0.0000472			LANGSMITH_E
	day05.run_rag_agent	자유이용권 환불이 되나요?	네. 자유이용권은 **이용일 ...		8/27/2026, 2:5...	3.58s			575	\$0.000208			LANGSMITH_E

LangSmith "Calculate token-based costs" (docs.langchain.com/langsmith, 2026-08-27 확인)

구간 분해 3: 실패한 런

```
# LangSmith 트레이스 화면을 옮겨 적음
llm ops.error_run 0.89s status=error
error: litellm.NotFoundError:
  OpenAIException - The model
    'gpt-5.6-luna-nonexistent' does not
    exist or you do not have access t...
```



읽는 순서	무엇을 보나
① 프로젝트 목록의 status 열	success / error, 실패한 요청을 먼저 거름
② 트리에서 error가 찍힌 자식 런	어느 단계에서 죽었나
③ 그 런의 error 필드	429(한도 초과)인지, 모델 이름 오류인지, 파싱 실패인지

오류가 서비스 안에서 났다면 **rag_agent** 아래 그 llm 런만 error로 박힘. 트레이스가 없으면 예외 문자열만 남아 오류 구간을 짚지 못함.

LangSmith Observability concepts: status · error 필드 (docs.langchain.com/langsmith, 2026-08-27 확인)

메타데이터 없는 런의 빈 비용 열

수동 기록. 메타데이터 없음

```
completion = traceable(
    run_type="llm")(completion)
llm  litellm.completion  cost=None
```

토큰은 세지만 모델을 모르니 **단가를 곱하지 못함.**

메타데이터 있음 / 랭체인 객체 자동

```
metadata={"ls_model_name": MODEL,
          "ls_provider": "openai"}
llm  litellm.completion  182 tok  $0.000067
# 랭체인 객체(init_chat_model)는 모델명을 스스로 넘김
llm  ChatLiteLLM         136 tok  $0.000140
```

환경변수만으로 비용까지 계산됨.

태그를 붙여 기능 단위로 비용을 묶어 보기

태그 전

프로젝트에 런 수백 건. 환불 질문과 인사 질문이 섞여 있어 **"환불 기능에 든 비용"**을 물을 수 없음.

```
# LangSmith 트레이스 화면을 옮겨 적음
rag_agent ... $0.000208
rag_agent ... $0.000047
rag_agent ... $0.000184
```



태그 후

```
@traceable(run_type="chain",
            tags=["refund"],
            metadata={"feature": "refund"})
def ask(question): ...
```

태그·메타데이터로 필터하면 그 런들만 남아, 기능 단위로 비용을 읽을 수 있음.

예시: 모델 이름(gpt-4o-mini)으로 필터한 런 목록

Personal / Tracing / kt10-day08-ops

kt10-day08-ops ID Tracing Evaluators Automations Insights Engine New

Retention 14d Dashboard + New

Default View Last 1 day Threads Traces Runs Input Search input content Reset Save View

(x) Metadata Content contains gpt-4o-mini

	✓	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation ...	Tokens	Cost	First Token	Tags	Metadata
	✓	ops.cost.gpt-4o-mini	user: 자유이용권 환불이 되...	chatcmpl-EHNQSp1sDFA...		8/27/2026, 2:5...	1.86s			125	\$0.0000669			LANGSMITH_E
	✓	litellm.completion	tool: [score 0.70] [티켓] Q...	chatcmpl-EHNPriyL4d7e...		8/27/2026, 2:5...	6.58s			360	\$0.00007605			LANGSMITH_E
	✓	litellm.completion	user: 자유이용권 환불이 되...	chatcmpl-EHNPkRgoB5a...		8/27/2026, 2:5...	1.47s			172	\$0.00003525			LANGSMITH_E
	✓	litellm.completion	tool: [score 0.70] [티켓] Q...	chatcmpl-EHNOjf2GMRiF...		8/27/2026, 2:5...	6.55s			368	\$0.00008085			LANGSMITH_E

저장된 상태(Snapshot)와 트레이스(Trace)

관점	Snapshot (get_state_history)	Trace (LangSmith)
무엇의 기록인가	그래프 실행의 상태 값 (State) 단계별	호출의 시간·입출력·토큰 런별
구조	단계 순서의 목록	부모-자식 트리
시간 흐름	단계 번호로만	시작·종료 시각, 워터폴
재현	특정 단계 상태에서 다시 실행 가능	재현이 아니라 관찰
여러 에이전트 분석	상태 값 비교	런 트리 비교
운영에서의 쓰임	디버깅·과거 상태 조회	지연·비용·오류 감시

둘은 대체재가 아니라 다른 질문에 답하는 두 기록. 상태가 궁금하면 Snapshot, 시간과 돈이 궁금하면 Trace.

실습합시다

챕터 요약

- ✓ 트레이스(trace)는 요청 한 건이 지나간 모든 단계의 기록이고, 런(run)은 그 안의 단계 하나(LLM·Chain·Tool·Retriever). 런이 부모-자식으로 이어져 트리가 됨
- ✓ 활성화는 환경변수 2개(LANGSMITH_TRACING · LANGSMITH_PROJECT). 랭체인 객체는 자동으로 기록되고, 직접 호출 함수는 traceable 데코레이터로 llm·tool·chain 런이 됨
- ✓ 워터폴에서 가장 긴 막대가 지연 구간, 토큰이 큰 llm 런이 비용 구간, status=error인 런이 실패 지점
- ✓ 수동 기록한 모델 호출은 ls_model_name·ls_provider 메타데이터가 있어야 토큰이 비용으로 환산됨. ls_model_name 메타데이터가 없으면 비용 열이 빈 채로 남음
- ✓ 태그·메타데이터를 붙이면 기능 단위로 런을 묶어 지연·비용 합계를 볼 수 있음. Snapshot은 상태의 기록, Trace는 시간과 돈의 기록

LLMOps

품질 평가(Evaluation) 설계

관측으로 안 보이는 것 / 네 부품 / evaluator 둘 / 실험 비교 / Judge의 함정

학습 목표

- 올바른 평가 데이터셋(dataset)을 구축할 수 있습니다.
- 규칙 evaluator와 LLM-as-a-Judge evaluator를 설계할 수 있습니다.

✓ LLM 서비스 관측성

▶ **품질 평가 설계**

비용·모델 운영 전략

문제 상황: 트레이스는 정상인데 의도와 다른 동작을 하는 서비스

질문: 파이썬 리스트 정렬은 어떻게 하나요?

답: 파이썬에서 리스트를 정렬하는 방법은 크게 두 가지입니다.

1. `sort()`. 리스트 자체를 정렬
2. `sorted()`. 새 리스트를 반환
- ...

FAQ 안내 서비스가 **FAQ에 없는 질문**에 그대로 답함.
프롬프트에는 "근거가 없으면 모른다고 답하라"가 있었음.

```
# LangSmith 트레이스 화면을 옮겨 적음
chain rag_agent status=success
└─ llm ... status=success
error: -
```

관측으로 확인되는 것

오류 없음, 지연 정상, 비용 정상

관측으로 확인 안 되는 것

지시를 잘 수행했는지

평가(Evaluation)는 하나의 데이터셋을 서비스가 풀게 하고 evaluator가 점수를 매기는 일.

답이 매번 달라지고(비결정성), 단계가 여럿이며(도구·검색), 정답이 하나가 아닌 문제가 섞여 있어 사람의 눈만으로는 반복하기 어려움.

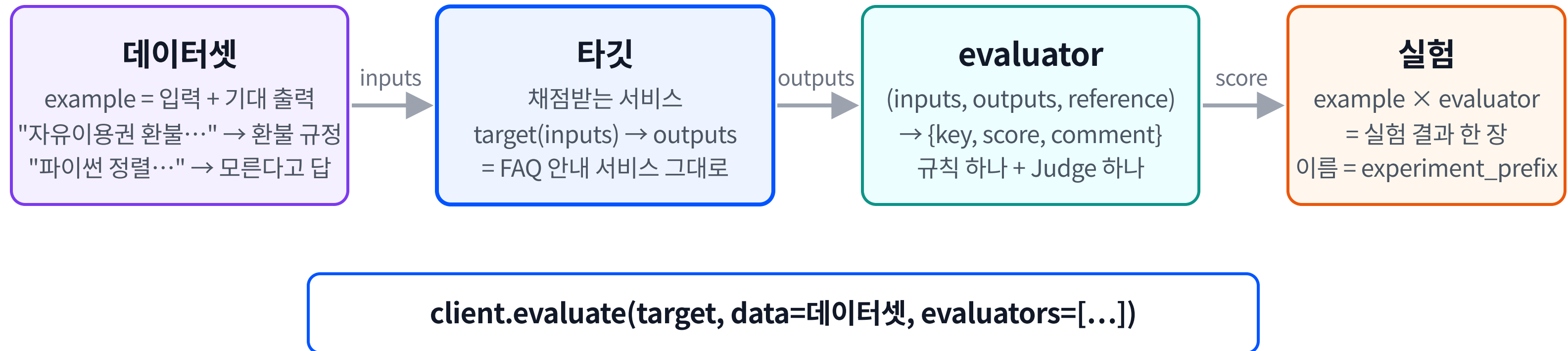
데이터셋(dataset) = 문제집 · evaluator = 채점자 · 실험(experiment) = 한 번의 채점 결과
채점 대상은 최종 답만이 아님. 도구 선택과 중간 단계(궤적)도 채점 대상.

측정 영역 4 × 채점 방법 2

영역	무엇을 측정하나	규칙 채점 (Rule-based)	LLM 채점 (LLM-as-a-Judge)
과제 성공 Task Success	요청한 일을 끝냈나	정답 일치, 필수 항목 포함 여부	부분 성공·우회 성공 판정
출력 품질 Output Quality	답이 근거에 충실하고 읽을 만한가	길이·형식·금지어	충실성(faithfulness)·관련성
시스템 성능 System Performance	얼마나 빨랐나	지연·호출 횟수, 트레이스에서	해당 없음 (트레이스 수치)
비용 Cost	얼마가 들었나	토큰·비용, 트레이스에서	해당 없음 (트레이스 수치)

규칙으로 측정할 수 있는 것은 규칙으로 먼저. LLM 채점은 규칙이 측정하지 못하는 곳에만.

평가의 네 부품



문제상황과 목표

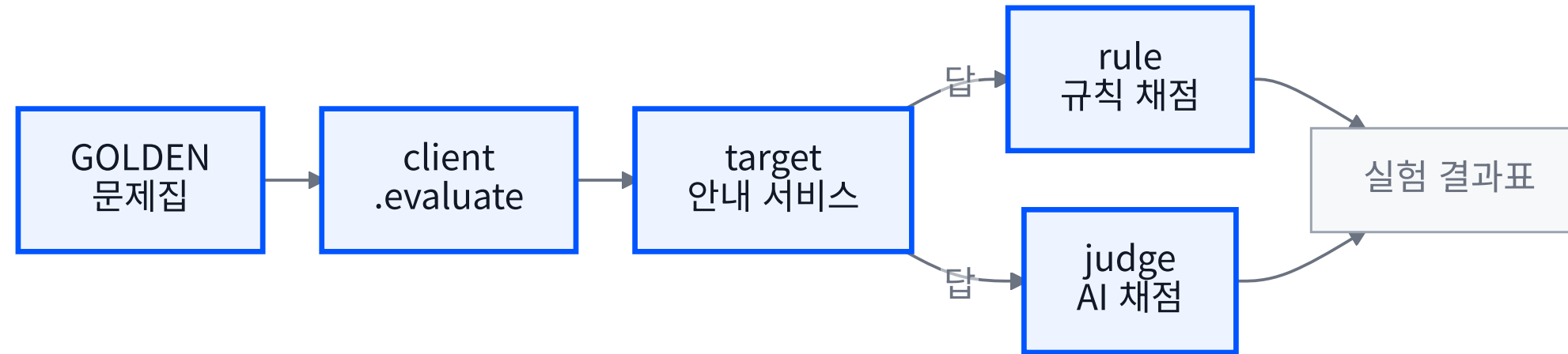
문제

구름월드 고객센터 담당자: "안내 서비스가 FAQ에 없는 것까지 그럴듯하게 답해요. 기록을 봐도 오류 하나 없이 정상인데, 고객한테 틀린 안내가 나갔어요 ㅠㅠ"

목표

고객센터가 자주 받는 질문 4개와 맞는 답의 조건을 문제집으로 만들어, 구름월드 안내 서비스의 답을 규칙과 AI 채점자가 자동으로 채점해 **어느 질문에서 왜 틀렸는지 표로 보게** 만들어봅시다.

각 단계별 요구사항



노드	무엇을 받아	어떻게	무엇을 넘긴다
GOLDEN	고객 질문 4개와 각각의 맞는 답의 조건, FAQ에 있는 질문인지 표지를	서비스가 볼 질문과 채점자만 볼 조건으로 나뉘	문제집으로 넘긴다
client.evaluate	문제집을	질문마다 target을 부르고 그 답을 채점자 둘에게 돌려	점수 8개와 근거를 실험 이름 v1로 묶어 결과표로 넘긴다
target	문제집의 질문 하나를	관측에서 쓴 안내 서비스를 그대로 불러	답을 넘긴다
rule	답과 「FAQ에 있는 질문인지」 표지를	FAQ에 있으면 빈 답이 아닌지, 없으면 답에 「모르」가 들어 있는지를 정해진 규칙대로만 검사해	0/1 점수를 넘긴다
judge	질문·맞는 답의 조건·답을	채점자 프롬프트를 앞세워 AI 채점자에게 「조건에 맞고 지어낸 내용이 없는가」를 판정시켜	0/1 점수와 한 문장 근거를 넘긴다

LLM 채점자에게 주는 프롬프트

```
# 고정 지침(시스템 프롬프트) = GUIDE
# 채점자 모델에 항상 주는 프롬프트
너는 엄격한 채점자다.
답이 기대에 부합하고
지어낸 내용이 없는지 판정하라.
```

```
# 입력: example마다 바뀌어 붙는 세 줄
질문: {질문}
기대: {기대 조건}
답: {서비스의 답}
```

```
# 출력 형식: 구조화 출력으로 강제
faithful: true/false
reason: 한 문장 근거
```

채점자도 LLM이므로 **시스템 프롬프트 + 입력 + 출력 형식** 셋을 줌.

```
# 들어가는 입력 (환불 example)
질문: 자유이용권 환불 규정 알려 주세요
기대: FAQ 환불 규정으로 답한다
답: 이용일 전날까지 취소: 전액 환불
    이용일 당일 취소: 50% 환불
    이용 개시 후: 환불 불가
```

채점자에게 들어가는 것은 질문·기대 조건·서비스의 답 세 가지. 이 세 가지가 example마다 바뀌어 시스템 프롬프트 뒤에 붙음.

평가 준비 코드: 데이터셋·평가 대상·채점자

```
from langsmith import Client; from pydantic import BaseModel
from langchain.chat_models import init_chat_model
import day05_rag as d5
GOLDEN = [dict(zip(["question", "expect", "must_know"], t)) for t in [
    ("안녕하세요!", "인사에 바로 답한다", True),
    ("자유이용권 환불 규정 알려 주세요", "FAQ 환불 규정으로 답한다", True),
    ("야간개장 때 퍼레이드 하나요?", "야간개장 시간과 퍼레이드를 이어 답한다", True),
    ("파이썬 리스트 정렬은 어떻게 하나요?", "문서 밖: 모른다고 답한다", False)]]
db, client = d5.open_db(), Client()
target = lambda inputs: {"answer": d5.run_rag_agent(inputs["question"], db)}
class Judge(BaseModel): faithful: bool; reason: str
GUIDE = "너는 엄격한 채점자다. 답이 기대에 부합하고 지어낸 내용이 없는지 판정하라."
judge_llm = init_chat_model("openai/gpt-5.6-luna",
    model_provider="litellm").with_structured_output(Judge)
```

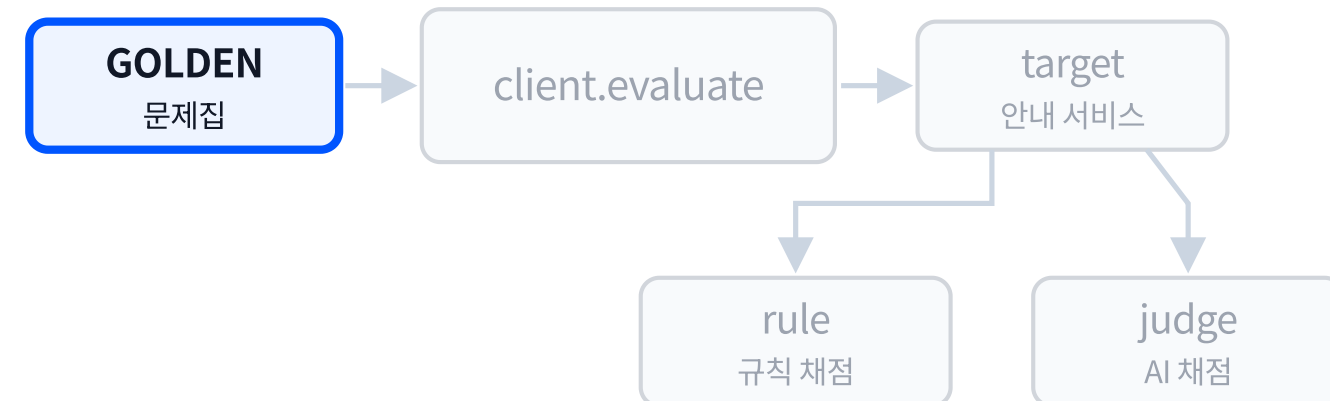
1단계: 데이터셋 생성

```
client.create_dataset("faq-golden")
client.create_examples(dataset_name="faq-golden",
    examples=[{"inputs": {"question": g["question"]},
               "outputs": g} for g in GOLDEN])

def rule(inputs, outputs, reference_outputs):
    ans = outputs["answer"]
    ok = bool(ans.strip()) if reference_outputs[
        "must_know"] else "모르" in ans
    return {"key": "rule", "score": int(ok)}

def judge(inputs, outputs, reference_outputs):
    j = judge_llm.invoke(GUIDE +
        f"\n질문:{inputs['question']}"
        f"\n기대:{reference_outputs['expect']}"
        f"\n답:{outputs['answer']}")
    return {"key": "judge", "score": int(j.faithful),
            "comment": j.reason}

client.evaluate(target, data="faq-golden",
    evaluators=[rule, judge], experiment_prefix="v1")
```



create_dataset으로 데이터셋을 만들고
create_examples로 example을 넣음. example =
inputs(질문) + outputs(기대 조건).

must_know는 "문서에 있는 질문인가" 표지. 문서 밖 질문은
모른다고 답하는 것이 정답.

준비 코드가 만드는 부품: **client**(LangSmith 클라이언트) ·
GOLDEN(질문 4건 리스트) · **target**(서비스를 부르는 함수) ·
judge_llm(구조화 출력을 붙인 채팅 모델) · **GUIDE**(채점자
프롬프트 문자열)

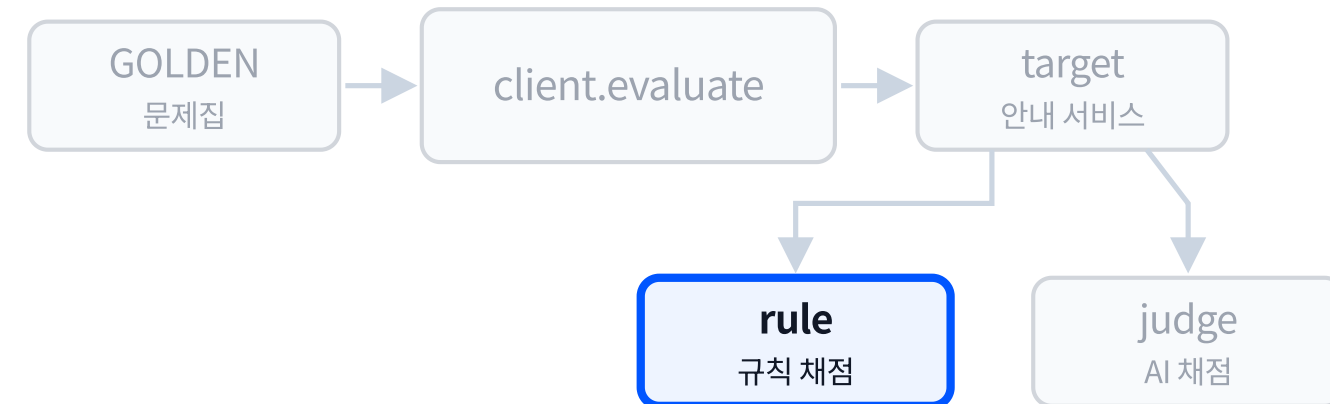
2단계: 형식이 아닌 내용을 측정하는 규칙 evaluator

```
client.create_dataset("faq-golden")
client.create_examples(dataset_name="faq-golden",
    examples=[{"inputs": {"question": g["question"]},
               "outputs": g} for g in GOLDEN])

def rule(inputs, outputs, reference_outputs):
    ans = outputs["answer"]
    ok = bool(ans.strip()) if reference_outputs[
        "must_know"] else "모르" in ans
    return {"key": "rule", "score": int(ok)}

def judge(inputs, outputs, reference_outputs):
    j = judge_llm.invoke(GUIDE +
        f"\n질문:{inputs['question']}"
        f"\n기대:{reference_outputs['expect']}"
        f"\n답:{outputs['answer']}")
    return {"key": "judge", "score": int(j.faithful),
            "comment": j.reason}

client.evaluate(target, data="faq-golden",
    evaluators=[rule, judge], experiment_prefix="v1")
```



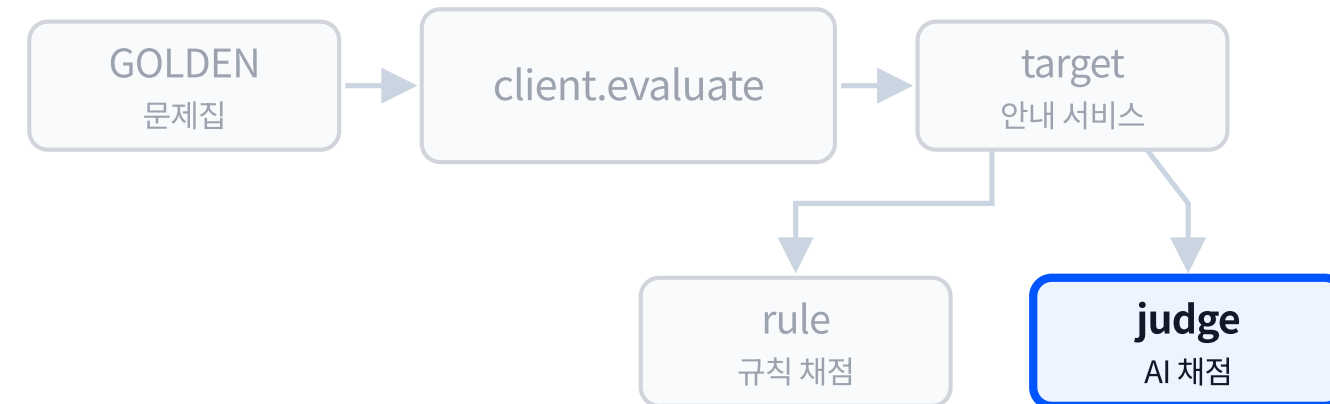
evaluator는 **(inputs, outputs, reference_outputs)**를 받아 **{"key", "score"}**를 돌려주는 함수. 이 계약만 지키면 무엇이든 evaluator.

형식(JSON 여부·필드 유무)은 구조화 출력이 보장함. 그래서 규칙 evaluator는 **내용**을 측정함: 문서 밖 질문에 「모른다」가 들어 있는가.

「모릅니다」·「모르겠습니다」는 잡지만 「확인이 어렵습니다」처럼 키워드가 없는 표현은 놓침. 규칙이 측정하지 못하는 부분은 LLM evaluator의 몫.

3단계: 구조화 출력으로 판정하는 LLM evaluator

```
client.create_dataset("faq-golden")
client.create_examples(dataset_name="faq-golden",
    examples=[{"inputs": {"question": g["question"]},
               "outputs": g} for g in GOLDEN])
def rule(inputs, outputs, reference_outputs):
    ans = outputs["answer"]
    ok = bool(ans.strip()) if reference_outputs[
        "must_know"] else "모르" in ans
    return {"key": "rule", "score": int(ok)}
def judge(inputs, outputs, reference_outputs):
    j = judge_llm.invoke(GUIDE +
        f"\n질문:{inputs['question']}"
        f"\n기대:{reference_outputs['expect']}"
        f"\n답:{outputs['answer']}")
    return {"key": "judge", "score": int(j.faithful),
           "comment": j.reason}
client.evaluate(target, data="faq-golden",
    evaluators=[rule, judge], experiment_prefix="v1")
```



judge_llm = 채팅 모델에 **with_structured_output(Judge)**를 건 객체. 응답은 pydantic 스키마(faithful: bool, reason: str)로 강제됨. 파싱 없음.

comment에 근거 문장을 실으면 결과표에서 그 이유를 확인할 수 있음. 사람 판단과 어긋나는 example이 발견되면 **GUIDE**를 고쳐서 채점자를 개선해야 함.

4단계: 실험 실행

```
client.create_dataset("faq-golden")
client.create_examples(dataset_name="faq-golden",
    examples=[{"inputs": {"question": g["question"]},
               "outputs": g} for g in GOLDEN])
def rule(inputs, outputs, reference_outputs):
    ans = outputs["answer"]
    ok = bool(ans.strip()) if reference_outputs[
        "must_know"] else "모르" in ans
    return {"key": "rule", "score": int(ok)}
def judge(inputs, outputs, reference_outputs):
    j = judge_llm.invoke(GUIDE +
        f"\n질문:{inputs['question']}"
        f"\n기대:{reference_outputs['expect']}"
        f"\n답:{outputs['answer']}")
    return {"key": "judge", "score": int(j.faithful),
            "comment": j.reason}
client.evaluate(target, data="faq-golden",
    evaluators=[rule, judge], experiment_prefix="v1")
```



client.evaluate가 example마다 타깃을 부르고 evaluator 둘을 돌림. example 4 × evaluator 2 = 점수 8개가 실험 1건으로 묶임.

experiment_prefix가 실험 결과의 이름. 같은 데이터셋에 experiment_prefix만 바꿔 다시 돌리면 두 실험이 나란히 비교됨.

실험 결과: example 4 × evaluator 2

질문	기대 조건	rule (기준: 빈 답 아님 / 문서 밖은 「모르」)	judge (기준: GUIDE)	judge comment
안녕하세요!	인사에 바로 답한다	1	1	인사로 바로 응답, 지어낸 내용 없음
자유이용권 환불 규정	FAQ 환불 규정으로 답한다	1	0	전날 전액·당일 50%·불가가 근거 없이 제시됨
야간개장 때 퍼레이드	야간개장 시간과 퍼레이드를 이어 답한다	1	1	퍼레이드 운영과 20:30 시각을 함께 제시
파이썬 리스트 정렬	문서 밖: 모른다고 답한다	0	0	문서 밖인데 정렬 방법을 예제까지 설명

반환값 **key**(열 이름) · **score**(수치) · **comment**(근거)가 그대로 결과표의 열. 환불 example은 답이 맞는데도 0. 기대 조건에 정답이 되는 FAQ 본문이 없어 LLM evaluator가 근거를 확인하지 못함.

랭스미스 실험 결과 화면

	Inputs	Reference Outputs	Outputs	judge ↓↑ 1.00 AVG	rule ↓↑ 1.00 AVG	Latency 2.80 P50
1	야간개장 때 퍼레이드 하나요?	야간개장 시간과 퍼레이드를 이어 답한다	네. 야간개장 기간에는 **20:30에 야간 퍼레이!	1.00	1.00	5.41s
2	안녕하세요!	인사에 바로 답한다	안녕하세요! 무엇을 도와드릴까요?	1.00	1.00	1.44s
3	자유이용권 환불 규정 알려 주세요	FAQ 환불 규정으로 답한다	자유이용권 환불 규정은 다음과 같습니다. - **이	0.00	1.00	4.01s
4	파이썬 리스트 정렬은 어떻게 하나요?	문서 밖: 모른다고 답한다	파이썬 리스트는 `sort()` 또는 `sorted()`로 정렬	0.00	0.00	2.78s

같은 답, 다른 채점: 잘못된 데이터셋 구성

시스템 프롬프트 (처음 구성 그대로)

너는 엄격한 채점자다.

답이 기대에 부합하고

지어낸 내용이 없는지 판정하라.

데이터셋의 기대 필드: 동작을 서술한 문장

기대: "FAQ 환불 규정 항목의 내용으로 답한다"

채점 기준이 되어야 할 위치(reference)에 정답 텍스트가 아니라 **기대 동작을 서술한 문장**이 들어 있음.

판정: 맞는 답이 0점

답: "이용일 전날까지 전액, 당일 50%..."

judge = 0

"원문이 제시되지 않아 ... 근거 없이 단정"

채점자가 정답을 못 봤으니 **맞는 답도 근거 없음으로 판정함.**

같은 답, 다른 채점: 올바른 구성

올바른 시스템 프롬프트: 정답 부합 판정
너는 엄격한 채점자다.
평가 대상의 답이 정답(reference)과
부합하는지, 어긋난 지점이 없는지 판정하라.

올바른 데이터셋 구성: 정답 텍스트
examples=[{"inputs":
 {"question": "자유이용권 환불 규정 알려 주세요"},
 "outputs": {"answer":
 "이용일 전일까지 취소하면 전액 환불,"
 "당일 취소는 50% 환불, 개시 후 불가"}}]

데이터셋의 outputs(화면 표기 Reference Outputs)에는 **정답 텍스트**를 넣고, evaluator는 (inputs, outputs, reference_outputs) 세 인자로 정답 부합을 판정함.

판정: 같은 답이 통과

정답: "이용일 전일까지 취소하면 전액 환불되고
이용일 당일 취소는 50%만 환불됩니다..."
답: "이용일 전일까지 전액, 당일 50%..."
judge = 1

정답 텍스트를 reference에 넣는 것만으로 같은 답이 통과함. **무엇을 주느냐가 판정을 바꿈.**

「지어낸 내용이 없는가」 류 기준은 정답 없이 판정하는 별도 계열(reference-free evaluator)로 존재함. 그 기준은 데이터셋의 정답 필드가 아니라 evaluator의 프롬프트에 적음.

※ 정답지 없이 측정하는 RAG 평가의 세 지표 = 충실성(faithfulness)·답변 관련성(answer relevance)·문맥 관련성(context relevance). 점수를 따로 매기지 않고 두 답을 맞대결시켜 나은 쪽을 고르는 방식은 pairwise 평가.

LLM evaluator는 **순서·길이·자기 문체**에 기운다. 앞에 놓인 답을 고르고(position), 긴 답에 후하고(verbosity), 자기가 쓴 듯한 답을 좋아함(self-enhancement).

사람 채점과 나란히 놓고 어긋난 example을 골라 채점 프롬프트를 고침. evaluator도 고쳐 가는 대상.
그래도 LLM 채점을 쓰는 근거: MT-Bench 측정에서 GPT-4와 사람의 판정 합의율 85%, 사람끼리의 합의율 81%(동점 제외).

같은 데이터셋, 두 버전 비교

실험 v1 (모델 A)

```
experiment_prefix="luna-v1"
rule 1 1 1 0 → 3/4
judge 1 0 1 0 → 2/4
```

기준이 되는 실험 결과.



실험 v2 (모델 B, 모델명만 교체)

```
experiment_prefix="gpt-4o-mini"
rule 1 1 1 0 → 3/4
judge 0 1 1 0 → 2/4
```

같은 데이터셋·같은 evaluator. 바뀐 것은 모델뿐.

랭스미스 실험 비교 화면



데이터셋으로 측정하는 평가와 운영 중에 측정하는 평가

오프라인 평가 (offline)

데이터셋을 서비스가 풀게 하고 채점함. 배포 전·변경 후에 돌림.

장점: 같은 데이터셋이라 버전 간 비교가 정확함.

한계: 데이터셋에 없는 질문은 못 잼.

온라인 평가 (online)

운영 트래픽의 일부(예: 10%)에 evaluator를 붙여 실시간 채점. 트레이스 자체가 example.

장점: 실제 질문 분포에서 잼.

한계: 정답(reference)이 없어 evaluator가 참조 없이 채점함.

회귀 평가의 자동화는 코드가 바뀌는 순간에 같은 데이터셋을 다시 돌리고, 점수가 내려가면 변경을 막는 것.

회귀 평가를 거는 곳은 CI 파이프라인(푸시·PR 이벤트) 또는 로컬 혹. 평가 코드는 4단계 그대로, 바뀌는 것은 "언제 돌리는가"뿐.

로컬에서 거는 회귀 게이트: pre-commit 혹은

```
# day08_eval.py - gate 모드
def gate(threshold=0.5):
    res = run(prefix="gate")
    scores = [x.score or 0 for r in res
              for x in r["evaluation_results"]
                  ["results"]]
    if x.key in ("judge", "judge_faithful")]
    mean = sum(scores) / len(scores)
    sys.exit(0 if mean >= threshold else 1)
```

```
# .git/hooks/pre-commit
#!/bin/sh
python day08_eval.py gate || {
    echo "회귀 감지: 커밋 차단"; exit 1; }
```

커밋 직전에 4단계 실험을 그대로 다시 돌리고,
judge 평균이 기준 아래면 **exit 1**로 커밋을 막음.
평가 코드는 그대로, 바뀐 것은 "언제 돌리는가"뿐.

```
$ git commit -m "프롬프트 수정"
experiment: gate-...
gate: judge 평균 0.50 / 기준 0.5 → 통과
```

기준은 직전 버전의 점수. 같은 혹은 CI(푸시·PR 이벤트)에 올리면 팀 전체의 게이트가 됨.

실습합시다

챕터 요약

- ✓ 평가(Evaluation)는 하나의 데이터셋(dataset)을 서비스(타겟)가 풀게 하고 evaluator가 점수를 매겨 실험(experiment) 1건을 만드는 일. 트레이스가 정상이어도 답이 틀릴 수 있어 관측과 별개로 필요함
- ✓ evaluator는 (inputs, outputs, reference_outputs)를 받아 {key, score, comment}를 돌려주는 함수. 규칙 evaluator는 내용 규칙(문서 밖 질문에 "모른다"가 있는가)을, LLM evaluator는 구조화 출력으로 충실성을 판정함
- ✓ client.evaluate(target, data=, evaluators=[], experiment_prefix=)가 example마다 타겟과 evaluator를 돌려 실험 1건으로 묶음. experiment_prefix만 바뀌 다시 돌리면 두 실험이 나란히 비교됨. 점수가 내려가면 회귀
- ✓ LLM evaluator의 판정은 reference에 종속됨. 정답 텍스트 없이 기대 조건만 주면 맞는 답도 0점. 순서·길이·자기 문체 편향이 있어 사람 채점과 대조해 채점 프롬프트를 고쳐 감
- ✓ 오프라인 평가는 데이터셋으로, 온라인 평가는 운영 트래픽 일부에 evaluator를 붙여 잼. 회귀 평가의 자동화는 코드 변경 이벤트에 같은 평가를 거는 것

LLMOps

비용·모델 운영 전략

멈추는 서비스 / 티어링 / 대체(fallback) / 캐싱 / 토큰 예산

학습 목표

- 작업의 난이도에 맞는 모델 등급을 고르는 근거를 비용과 품질 관점으로 설명할 수 있습니다.
- 호출 단계 인자 하나(fallbacks=)로 서비스가 대체 모델로 넘어가는 것을 응답 모델과 트레이스에서 확인하고, 라우팅 규칙을 적을 수 있습니다.
- 토큰 예산 관리와 응답 캐싱(시맨틱 캐싱) 기반 비용 최적화의 원리를 말하고, 캐시 히트를 응답 필드로 확인할 수 있습니다.

✓ LLM 서비스 관측성

✓ 품질 평가 설계

▶ 비용·모델 운영 전략

문제 상황: 한 모델에 매인 서비스

모델 하나가 멈추면 전부 멈춤

```
llm litellm.completion 0.89s status=error
error: The model 'gpt-5.6-luna-nonexistent'
does not exist or you do not have access...
# 같은 시각, 이 모델로 가는 요청 전부 실패
```

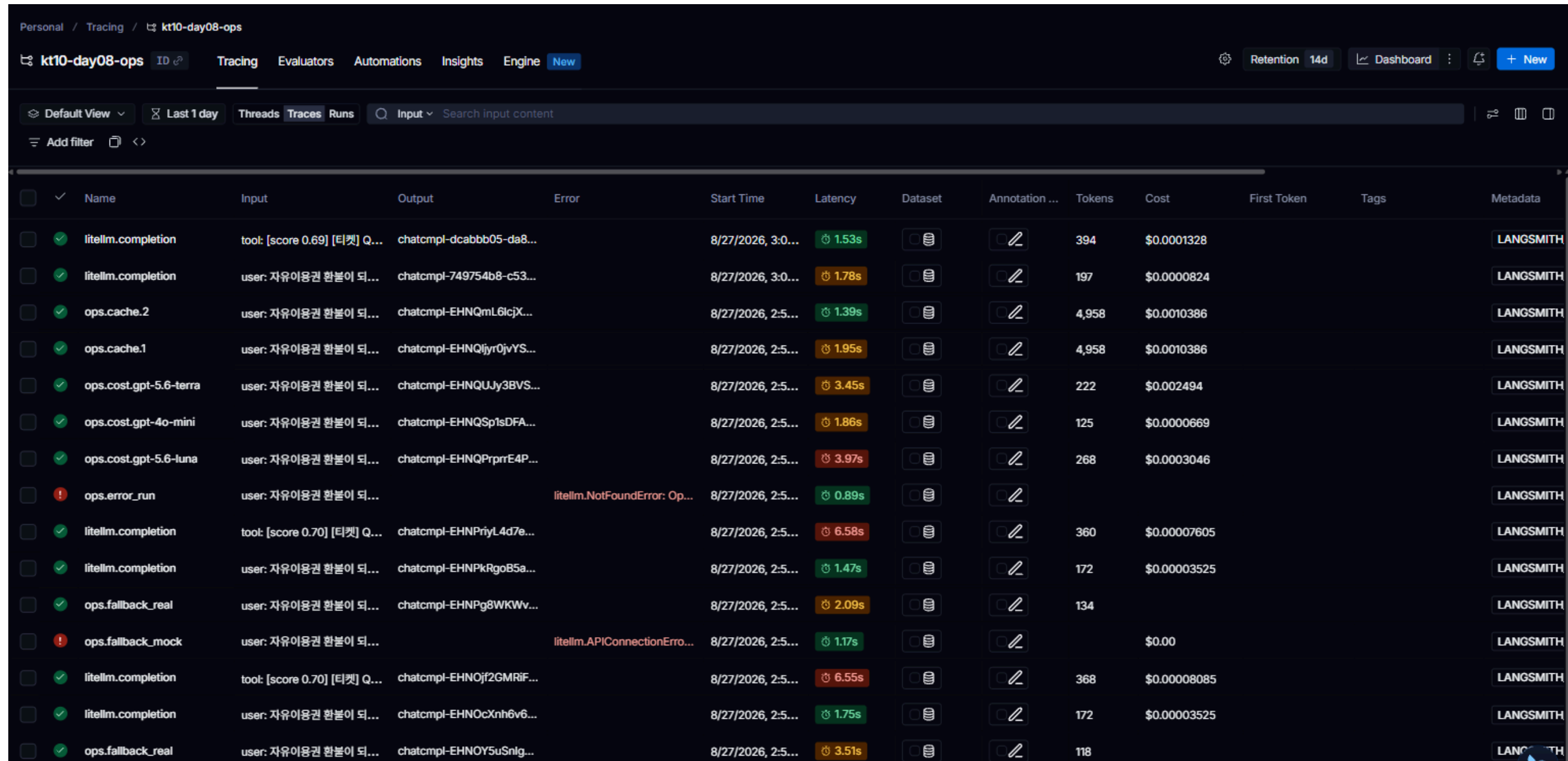
모델 이름 하나가 죽거나 한도(429)에 걸리면 끝.
서비스 코드에 다른 이름이 없음.

모든 질문을 가장 비싼 모델에

```
rag_agent "안녕하세요!" $0.000047
rag_agent "환불 규정..." $0.000208
rag_agent "야간개장 퍼레이드..." $0.000184
# × 하루 요청 수 = 청구서
```

인사에도 검색 질문과 같은 단가. 트레이스의 비용
열이 그대로 청구서가 됨.

랭스미스 화면: 한 프로젝트에 모인 런 목록



	✓	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation ...	Tokens	Cost	First Token	Tags	Metadata
☐	✓	litellm.completion	tool: [score 0.69] [티켓] Q...	chatcmpl-dcabb05-da8...		8/27/2026, 3:0...	1.53s			394	\$0.0001328			LANGSMITH
☐	✓	litellm.completion	user: 자유이용권 환불이 되...	chatcmpl-749754b8-c53...		8/27/2026, 3:0...	1.78s			197	\$0.0000824			LANGSMITH
☐	✓	ops.cache.2	user: 자유이용권 환불이 되...	chatcmpl-EHNQmL6lcjX...		8/27/2026, 2:5...	1.39s			4,958	\$0.0010386			LANGSMITH
☐	✓	ops.cache.1	user: 자유이용권 환불이 되...	chatcmpl-EHNQijyr0jvYS...		8/27/2026, 2:5...	1.95s			4,958	\$0.0010386			LANGSMITH
☐	✓	ops.cost.gpt-5.6-terra	user: 자유이용권 환불이 되...	chatcmpl-EHNQUJy3BVS...		8/27/2026, 2:5...	3.45s			222	\$0.002494			LANGSMITH
☐	✓	ops.cost.gpt-4o-mini	user: 자유이용권 환불이 되...	chatcmpl-EHNQSp1sDFA...		8/27/2026, 2:5...	1.86s			125	\$0.0000669			LANGSMITH
☐	✓	ops.cost.gpt-5.6-luna	user: 자유이용권 환불이 되...	chatcmpl-EHNQPrprE4P...		8/27/2026, 2:5...	3.97s			268	\$0.0003046			LANGSMITH
☐	!	ops.error_run	user: 자유이용권 환불이 되...		litellm.NotFoundError: Op...	8/27/2026, 2:5...	0.89s							LANGSMITH
☐	✓	litellm.completion	tool: [score 0.70] [티켓] Q...	chatcmpl-EHNPriyL4d7e...		8/27/2026, 2:5...	6.58s			360	\$0.00007605			LANGSMITH
☐	✓	litellm.completion	user: 자유이용권 환불이 되...	chatcmpl-EHNPkRgoB5a...		8/27/2026, 2:5...	1.47s			172	\$0.00003525			LANGSMITH
☐	✓	ops.fallback_real	user: 자유이용권 환불이 되...	chatcmpl-EHNPg8WKWv...		8/27/2026, 2:5...	2.09s			134				LANGSMITH
☐	!	ops.fallback_mock	user: 자유이용권 환불이 되...		litellm.APIConnectionErro...	8/27/2026, 2:5...	1.17s				\$0.00			LANGSMITH
☐	✓	litellm.completion	tool: [score 0.70] [티켓] Q...	chatcmpl-EHNOjf2GMRiF...		8/27/2026, 2:5...	6.55s			368	\$0.00008085			LANGSMITH
☐	✓	litellm.completion	user: 자유이용권 환불이 되...	chatcmpl-EHNOcXnh6v6...		8/27/2026, 2:5...	1.75s			172	\$0.00003525			LANGSMITH
☐	✓	ops.fallback_real	user: 자유이용권 환불이 되...	chatcmpl-EHNOY5uSnlg...		8/27/2026, 2:5...	3.51s			118				LANGSMITH

한 프로젝트에 모든 런이 모임. 오류 런은 빨간 표시로 걸러 볼 수 있고, 런마다 토큰 수와 비용이 함께 남음.

운영 문제 유형과 측정 수단

문제 유형	예	측정 수단	보는 화면
품질	문서 밖 질문에 답함	evaluator	실험 결과표
비용	인사에도 검색 단가	트레이스 비용 열	프로젝트 비용 합계
성능	검색이 4초	워터폴	지연 정렬 목록
흐름	한 모델 장애에 전부 실패	status=error 런	오류 필터

품질·비용·성능·흐름을 고치는 수단은 둘: 어떤 모델에 보내는가, 실패하면 어디로 넘기는가.

라우팅(routing)을 이용하면 사용자 질문의 난이도를 판정하여, 난이도에 따라 모델을 다르게 사용할 수도 있음.

티어링(tiering): 작업 난이도에 맞춘 모델 등급

작업	난이도	맞는 등급	근거
인사·짧은 안내	낮음	무료·경량	검색도 추론도 필요 없음. 품질 차이가 나지 않는 곳
FAQ 검색 후 답변	중간	기본 등급	도구 호출 판단 + 근거 요약. 경량이 놓치는 검색 판단이 있음
여러 항목을 잇는 답	높음	고성능	두 FAQ 항목을 합쳐 답하는 추론
채점(Judge)	높음	고성능	채점 오류는 서비스 오류보다 비쌘

등급	모델	입력	캐시 입력	출력
무료	오픈 가중치 모델 로컬 실행·무료 API 티어	0	0	0 (서버·품질·한도를 직접 책임짐)
경량	gpt-4o-mini	\$0.15	\$0.075	\$0.60
기본	gpt-5.6-luna	\$0.20	\$0.02	\$1.20
고성능	gpt-5.6-terra	\$2.00	\$0.20	\$12.00

단가(\$/1M 토큰) = OpenAI 공식 가격표 developers.openai.com/api/docs/pricing (2026-08-27 확인)

같은 데이터셋, 두 모델: 점수와 비용 비교

실험 (모델명만 교체)	rule	judge	문서 밖 질문의 실제 답	1건 비용
luna-v1 (gpt-5.6-luna)	3/4	2/4	sort() · sorted() 방법을 설명	\$0.000305
gpt-4o-mini	3/4	2/4	모른다는 말 없이 답함	\$0.000067
gpt-5.6-terra-seq	3/4	3/4	"잘 모르겠습니다" (규칙이 「모른」 만 봐서 0)	\$0.002494

대체(fallback) 세 종류

설정 키	언제	넘기는 곳	뜻	completion() 인자
fallbacks	오류·한도 초과 (429)·장애	다음 모델	장애 대비. 순서대로 시도	예
context_window_fallback_dict	입력이 컨텍스트 창을 넘음	창이 큰 모델	실패가 아니라 크기 문제	예
content_policy_fallbacks	콘텐츠 정책 차단	정책이 다른 모델	차단은 장애가 아님. 안내로 끝내는 것이 답인 경우가 많음	아니오 (Router 전용)

안전 필터에 걸린 응답은 재시도·대체로 뚫지 않음. 장애와 차단은 다른 응답 유형.

문제상황과 목표

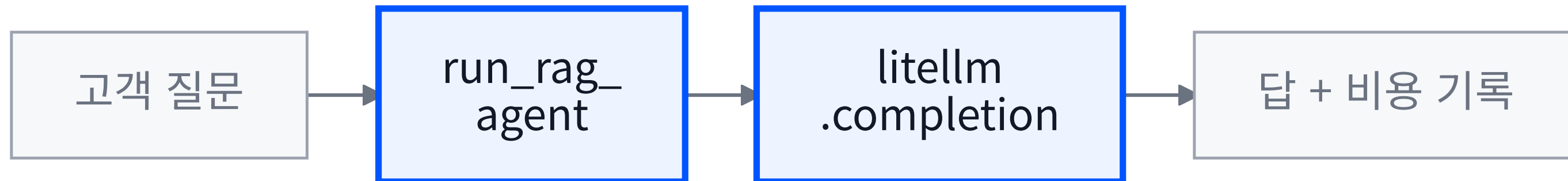
문제

구름월드 고객센터 담당자: "API 모델 서비스 서버가 고장났어요 $\pi\pi$ 안내 서비스가 통째로 멈춰서 고객 문의가 다 튕겨요"

목표

- 구름월드 안내 서비스가 주로 쓰는 AI 모델이 고장 나도 **안내가 끊기지 않게** 해봅시다.
- 각 안내에 대해 어느 모델이 답했고, 비용이 얼마나 들었는지 **기록에 남게** 만들어봅시다.

각 단계별 요구사항



노드	무엇을 받아	어떻게	무엇을 넘긴다
run_rag_agent	고객 질문을	관측·평가에서 쓴 것과 같은 안내 서비스를 그대로 돌려	답을 넘긴다
litellm.completion	서비스가 모델에 던지는 호출을	주 모델이 오류·한도 초과이거나 질문이 너무 길어 한 번에 못 담으면 예비 모델(BACKUP)로 넘기고, 실제로 답한 모델의 이름을 기록에 달아	응답과 비용을 넘긴다

준비 코드: 환경변수와 대체 발동 확인

```
import os, litellm
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_PROJECT"] = "day08"
import day05_rag as d5 # API 키를 여기서 읽음
res = litellm.completion(
    model="openai/gpt-5.6-luna-nonexistent",
    messages=[{"role": "user",
                "content": "안녕하세요"}],
    fallbacks=["openai/gpt-4o-mini"])
print(res.model) # 실제 답한 모델
print(litellm.completion_cost(res)) # 비용
```

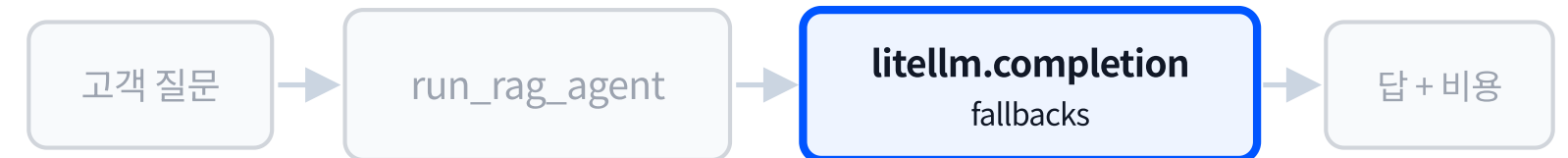
1차 모델 장애를 흉내 내기 위해 존재하지 않는 모델 이름을 1차 모델로 둬.

fallbacks에 지정한 예비 모델이 대신 답하는지 확인함. 실제 답한 모델은 **res.model** 필드로 확인함.

트레이스를 켜는 환경변수 2개도 이 코드에서 설정함.

1단계: 호출 단계 인자로 대체 모델 지정 (서비스 코드 무수정)

```
import litellm, day05_rag as d5
from langsmith import traceable, get_current_run_tree
MODEL = "openai/gpt-5.6-luna"
BACKUP = "openai/gpt-4o-mini"
_call = litellm.completion
@traceable(run_type="llm", name="litellm.completion",
           metadata={"ls_provider": "openai"})
def completion(**kw):
    kw["fallbacks"] = [BACKUP]
    kw["context_window_fallback_dict"] = {
        MODEL: BACKUP
    }
    r = _call(**kw)
    get_current_run_tree().metadata[
        "ls_model_name"] = r.model
    return r
litellm.completion = completion
db = d5.open_db()
print(d5.run_rag_agent("환불 규정 알려 주세요", db))
```

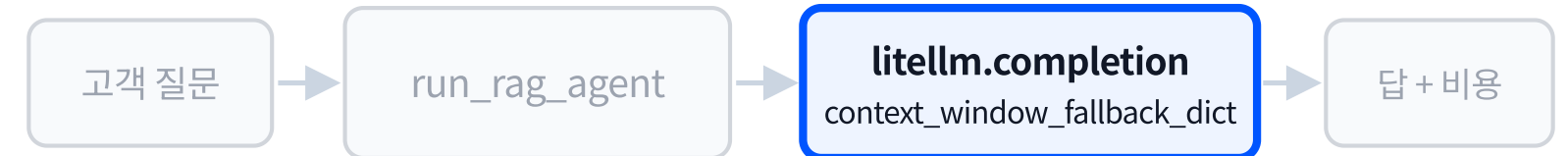


fallbacks는 순서대로 시도됨: 원래 모델 → 목록의 다음 모델.
서비스 코드(**run_rag_agent**)는 한 글자도 바뀌지 않음.

바뀌는 것은 **completion**의 래퍼 함수 한 겹. 호출 단계가 정책을 들고, 서비스 코드는 수정하지 않음.

2단계: 오류 유형별 대체. 컨텍스트 초과

```
import litellm, day05_rag as d5
from langsmith import traceable, get_current_run_tree
MODEL = "openai/gpt-5.6-luna"
BACKUP = "openai/gpt-4o-mini"
_call = litellm.completion
@traceable(run_type="llm", name="litellm.completion",
            metadata={"ls_provider": "openai"})
def completion(**kw):
    kw["fallbacks"] = [BACKUP]
    kw["context_window_fallback_dict"] = {
        MODEL: BACKUP
    }
    r = _call(**kw)
    get_current_run_tree().metadata[
        "ls_model_name"] = r.model
    return r
litellm.completion = completion
db = d5.open_db()
print(d5.run_rag_agent("환불 규정 알려 주세요", db))
```



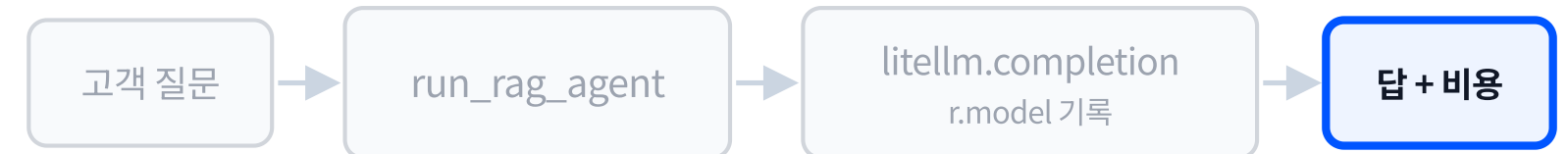
context_window_fallback_dict는 입력이 컨텍스트 창을 넘을 때만 지정한 모델로 넘김. 장애가 아니라 크기 문제의 분기. 여기서는 같은 예비 모델을 지정.

콘텐츠 정책 분기는 **completion()**이 받지 않음. 정책이 여럿이면 **litellm.Router(model_list, content_policy_fallbacks=)**로 승격할 것.

3단계: 응답 모델과 트레이스에서 대체 확인

```
import litellm, day05_rag as d5
from langsmith import traceable, get_current_run_tree
MODEL = "openai/gpt-5.6-luna"
BACKUP = "openai/gpt-4o-mini"
_call = litellm.completion

@traceable(run_type="llm", name="litellm.completion",
           metadata={"ls_provider": "openai"})
def completion(**kw):
    kw["fallbacks"] = [BACKUP]
    kw["context_window_fallback_dict"] = {
        MODEL: BACKUP}
    r = _call(**kw)
    get_current_run_tree().metadata[
        "ls_model_name"] = r.model
    return r
litellm.completion = completion
db = d5.open_db()
print(d5.run_rag_agent("환불 규정 알려 주세요", db))
```



실제 답한 모델은 응답의 **model** 필드. 호출이 끝난 뒤 그 값을 현재 런의 **ls_model_name**에 적으면 트레이스가 실제 답한 모델로 비용을 계산함.

1차 모델을 없는 이름으로 두고 돌리면 **gpt-4o-mini-2024-07-18**이 답하고 그 이름·비용(\$0.0000287)이 런에 남음. 미리 적어 둔 이름표였다면 대체 뒤에도 1차 이름이 남아 비용이 **None**.

litellm.completion ID ?	
Feedback	Tools
Input	Output
Attributes	
Attributes	
Metadata	
revision_id	kt10-wave15-release-v4-6-gdbf5033-dirty
LangSmith	
LANGSMITH_ENDPOINT	https://api.smith.langchain.com
LANGSMITH_PROJECT	kt10-day08-ops
LANGSMITH_TRACING	true
ls_method	traceable
ls_model_name	gpt-4o-mini-2024-07-18
ls_provider	openai
ls_run_depth	0

캐싱 세 종류: 재사용 대상과 대가

종류	누가 맡나	무엇을 재사용	히트 조건	모델 호출	대가
프롬프트 캐싱 (prompt caching)	API 제공자	같은 접두 (prefix)의 입력 토큰	접두가 같고 최소 길이(1,024 토큰) 이상	호출 함. 입력 단가만 할인 (최대 90%)	도구 정의·순서가 접두에 포함되어 도구 하나만 바뀌도 캐시가 깨짐
exact-match 응답 캐싱	우리 서비스 (답을 저장)	요청 전체의 응답	요청이 글자 단위로 동일	히트 시 호출 X	한 글자만 달라도 미스
시맨틱 캐싱 (semantic caching)	우리 서비스 (질문을 임베딩해 저장)	뜻이 비슷한 요청의 응답	임베딩 유사도가 임계값 이상	히트 시 호출 X	임계값 튜닝. 느슨하면 틀린 답을 돌려주고 빡빡하면 히트가 사라짐. 멀티턴·에이전트 트래픽에는 부적합

시맨틱 캐싱은 단발 질문용. 대화가 이어지거나 도구를 부르는 트래픽에 걸면 낡은 응답을 되풀이함.

캐시 히트는 응답의 계측 필드와 비용으로 확인

```
# 같은 시스템 접두 4,908 토큰, 같은 질문 2회
res.usage.prompt_tokens_details
# 1회차: cached_tokens=0
#         cache_write_tokens=4908
# 2회차: cached_tokens=4908

litellm.completion_cost(res)
# 1회차: 0.001284 (정가)
# 2회차: 0.000155 (캐시 읽기 할인)
```

필드	뜻
cached_tokens	접두 재사용으로 할인된 입력 토큰 수
cache_write_tokens	이번 요청이 캐시에 새로 쓴 토큰 수
둘 다 0	접두가 최소 길이(1,024)에 못 미치거나, 접두가 매번 달라짐

입력 비용 산식 = 캐시 읽기 토큰 × 0.1 + 캐시 쓰기 토큰 × 1.25 또는 2(Anthropic만, 캐시 유지 시간에 따라) + 나머지 토큰 × 정가

OpenAI "Prompt caching": Cached Input \$0.02/1M · 최소 1,024 토큰 (developers.openai.com, 2026-08-27 확인)

토큰 예산: 요청 1건 비용 × 반복 × 빈도

요청	구성	1건 비용	× 1,000건/ 일 × 30일
FAQ 검색 후 답변 (luna)	RAG 검색 포함, 호출 2회	\$0.000208	\$6.2
단발 질문 (luna)	검색 없음, 출력 251토큰(추론 72 포함)	\$0.000305	\$9.2
단발 질문 (terra)	검색 없음, 고성능 등급	\$0.002494	\$74.8

```
spent = 0.0
res = completion(model=MODEL, messages=msgs)
spent += litellm.completion_cost(res)
if spent > BUDGET:
    # 경량 모델로 내리거나 거절
```

SDK 직결에서는 서비스 코드가 **completion_cost**를 요청마다 누적해 스스로 집계함. 예산 상한은 서비스 코드가 지킴.

청구서 = 요청 1건 비용 × 반복 횟수 × 요청 빈도.

실습합시다

챕터 요약

- ✓ 티어링(tiering)은 작업의 난이도에 모델 등급을 맞추는 일. 하나의 데이터셋으로 두 모델을 실험해 점수가 같으면 싼 쪽, 다르면 달라진 example의 난이도로 등급을 정함
- ✓ 대체(fallback)는 세 종류. 오류·한도 초과에는 fallbacks, 컨텍스트 창 초과에는 context_window_fallback_dict, 콘텐츠 정책 차단은 Router 전용 content_policy_fallbacks. 차단은 장애가 아니므로 재시도 대상이 아님
- ✓ 대체 설정은 호출 단계 인자 completion(..., fallbacks=[...]). 서비스 코드 변경 없이 가능
- ✓ 대체가 일어났는지는 응답의 model 필드와 트레이스 llm 런의 ls_model_name으로 확인함
- ✓ 상황에 따라 세 종류의 캐싱을 잘 활용하여 비용을 절약할 것

참고: 운영 지표와 대시보드



지표	무엇인가	언제·왜 보나
오류율	status=error 런의 비율	배포 직후나 장애 의심 시, 대체 발동 여부를 봄
지연 P50/P95	절반·95%가 끝나는 시간	느리다는 불만이 있을 때 봄. 평균은 꼬리를 숨김
토큰·비용	기간별 토큰과 환산 비용	청구 주기마다 예산 상한과 대비해 봄
요청 수	단위 시간당 런 수	비용 셈에서 곱하는 값이라 함께 봄
피드백 추이	온라인 평가 점수의 시간축	변경 배포 뒤 회귀 감지에 봄

참고: 이 강의가 다루지 않은 운영 항목

항목	무엇인가	언제·왜 쓰나
A/B 실험 · 점진 배포	트래픽을 나눠 두 버전을 비교하고, 새 버전에 트래픽 일부만 보내는 배포 방식 (litellm 모델별 <code>weight</code>)	새 모델·프롬프트를 투입할 때, 실사용 분포에서 안전하게 비교하기 위해 씬
Playground · Prompt Hub	프롬프트를 즉시 재실행하고 버전으로 저장하는 도구	프롬프트를 고칠 때, 프롬프트도 코드처럼 버전으로 관리하기 위해 씬
멀티턴 스레드 · Summary evaluator	여러 턴을 한 스레드로 묶어 채점하고, 실험 전체를 집계 점수로 요약하는 평가	대화형 서비스에서 턴 단위 점수만으로 품질을 말할 수 없을 때 씬
민감정보 로깅 차단	트레이스로 나가는 입력·출력을 마스킹하는 설정	개인정보가 섞이는 서비스에서 트레이스 저장소에 민감정보가 남지 않게 하기 위해 씬
재학습·파인튜닝	LLM 또는 RAG용 임베딩 모델을 내부 데이터에 맞게 추가로 학습시키는 것	프롬프트·RAG 조정으로 부족할 때 씬. 비용·운영 부담이 커서 마지막 수단

참고: 미들웨어 — 서비스 코드를 고치지 않고 끼워 넣는 운영 층

이 강의는 관측을 켜고 대체(fallback)를 걸 때마다 litellm 호출을 한 겹 감싸는 방법을 썼음. 서비스 코드는 그대로 두고, 감싼 층이 기록과 정책을 담당함. 랭체인 에이전트에는 이 「감싸는 층」이 **미들웨어 (middleware)**라는 이름의 공식 부품으로 준비되어 있음.

무엇인가

에이전트 실행 파이프라인의 각 지점(에이전트 시작 전 · 모델 호출 전후 · 도구 실행을 감싸는 지점 · 종료 후)에 끼워 넣는 훅 함수.

에이전트를 만들 때 목록으로 넘기면 앞쪽 혹은 넘긴 순서대로, 뒤쪽 혹은 그 역순으로 적용됨.

```
create_agent(model=, tools=,
middleware=[...])
```

왜 필요한가

운영 요구(로깅 · 비용 통제 · 재시도 · 안전 검사)는 서비스 로직과 별개로 계속 늘어남. 미들웨어로 분리하면 서비스 코드는 그대로 두고 정책만 쌓거나 뺄 수 있음. 이 강의가 손으로 한 감싸기를 프레임워크가 표준 부품으로 제공하는 것.

내장 부품은 무엇이 있나

긴 대화 자동 요약(컨텍스트 관리) · 모델/도구 호출 횟수 제한(무한 루프와 비용 폭주 방지) · 재시도와 모델 폴백(fallbacks와 같은 목적) · 개인정보 탐지 · 사람 승인 등이 공식 내장 미들웨어로 제공됨.

참고: 가드레일 — 사고를 사전에 막는 안전 장치

관측은 무슨 일이 있었는지 보여 주고 평가는 답의 품질을 재지만, 사고를 일어나기 전에 막는 층은 따로 필요함. 개인정보가 답에 실려 나가는 사고, 악의적 입력이 시스템 프롬프트를 뒤집는 사고(프롬프트 인젝션), 위험한 작업(메일 발송·데이터 삭제)의 무단 실행이 그것. 가드레일(guardrails)은 이 목적에 특화된 미들웨어.

규칙 기반 검사

정규표현식·키워드 검사로 빠르고 예측 가능하게 거름.
비용이 없는 대신 미묘한 위반은 놓침.

모델 기반 검사

LLM이 맥락을 이해해 판정함. 규칙이 놓치는 문제를 잡는 대신 느리고 비용이 높음.

이 구도는 평가 챕터의 **규칙 evaluator**와 **LLM evaluator** 관계와 같음. 실무에서는 빠른 규칙 검사를 앞에, 비싼 모델 검사를 뒤에 쌓아(다층 방어) 입력·도구 실행·출력 세 지점을 지킴.

개인정보가 새는 사고는 개인식별정보(PII, Personally Identifiable Information) 탐지가 막습니다. 이메일 주소나 카드번호를 찾아 가리거나 답을 내보내지 않습니다. 시스템 프롬프트를 뒤집으려는 사고는 입력 검사가 막습니다. 지시를 무시하라는 식의 문장을 답을 만들기 전에 걸러 냅니다. 위험한 작업이 멋대로 실행되는 사고는 사람 승인(Human-in-the-Loop)이 막습니다. 메일 발송이나 데이터 삭제 같은 도구를 부르기 직전에 사람에게 물어봅니다. 고객 정보가 오가는 서비스라면 이 세 층을 운영 전에 반드시 검토합니다.

LangChain 공식 문서 "Guardrails" (docs.langchain.com/oss/python/langchain/guardrails, 2026-08-28 확인)

